Programming Styles

Style #23: Passive Aggressive

•••

Group 2
Amanda, Anitta, Nisa, Griffin, and Taqdeer

Summary & Constraints

Sanity Checking

- Functions validate all inputs
- Functions with invalid input stop immediately and refuse to continue (using assert statements or type checks)

Function Calling

- Functions only check for errors they can meaningfully react to
- Functions do not catch errors unnecessarily

Error Handling

- Occurs at higher levels of function call chains
- The monadic bind captures exception and stops execution

Input and Output of Program

Input	Output
White tigers live mostly in India Wild lions live mostly in Africa	live - 2 mostly - 2 white - 1 tigers - 1 india - 1 wild - 1 lions - 1 africa - 1
"Some acquaintance or other, my dear, I suppose; I am sure I do not know."	acquaintance - 1 suppose - 1 sure - 1 know - 1

Walkthrough of Python Code

```python
#!/usr/bin/env python
import sys, re, operator, string

#
# The PassiveAggressive class for this example
#
class TFPassiveAggressive:
    def __init__(self):
        self._e = None
        self._offending_func = None
        self._value = None

    def bind(self, func):
        if self._e == None:
            try:
                self._value = func(self._value)
            except Exception as e:
                self._e = e
                self._offending_func = func
        return self

    def printme(self):
        if self._e == None:
            print(self._value)
        else:
            print(self._e, " in ", self._offending_func.__name__)

```

__init__: initialises current pipeline.

bind(self, func): runs a function if there is no previous error.

- ➔ If self._e is None, it tries running func(self._value) and stores result in self._value.
- ➔ If a function throws an exception, save error in self._e and save function in self._offending_func.
- ➔ return self allows chaining/binding.

printme(): if no error, it prints the final self._value.

- ➔ If error, it prints the exception and name of the failing function.

Walkthrough of Python Code

```python
#
# The functions
#
def get_input(ignore):
    assert(len(sys.argv) > 1), "You idiot! I need an input file! I quit!"
    return sys.argv[1]

def extract_words(path_to_file):
    assert(type(path_to_file) is str), "I need a string! I quit!"
    assert(path_to_file), "I need a non-empty string! I quit!"

    with open(path_to_file) as f:
        data = f.read()
    pattern = re.compile('[\W_]+')
    word_list = pattern.sub(' ', data).lower().split()
    return word_list

def remove_stop_words(word_list):
    assert(type(word_list) is list), "I need a list! I quit!"

    with open('../stop_words.txt') as f:
        stop_words = f.read().split(',')
    # add single-letter words
    stop_words.extend(list(string.ascii_lowercase))
    return [w for w in word_list if not w in stop_words]

```

get_input(ignore): returns exception if input file does not exist

➔ otherwise returns input file

extract_words(path_to_file): returns list of words in input file if path_to_file is a non-empty string

➔ otherwise returns an exception

remove_stop_words(word_list): if word_list is a list, returns words from word_list (excluding words from stop_words.txt).

➔ If word_list is not a list, returns exception

Walkthrough of Python Code

```python
def frequencies(word_list):
    assert(type(word_list) is list), "I need a list! I quit!"
    assert(word_list != []), "I need a non-empty list! I quit!"

    word_freqs = {}
    for w in word_list:
        if w in word_freqs:
            word_freqs[w] += 1
        else:
            word_freqs[w] = 1
    return word_freqs

def sort(word_freqs):
    assert(type(word_freqs) is dict), "I need a dictionary! I quit!"
    assert(word_freqs != {}), "I need a non-empty dictionary! I quit!"

    return sorted(word_freqs.items(), key=operator.itemgetter(1), reverse=True)

def top25_freqs(word_freqs):
    assert(type(word_freqs) is list), "I need a list! I quit!"
    assert(word_freqs != {}), "I need a non-empty dictionary! I quit!"

    top25 = ""
    for tf in word_freqs[0:25]:
        top25 += str(tf[0]) + ' - ' + str(tf[1]) + '\n'
    return top25
```

frequencies(word_list): ensures input is a non-empty list, counts input list, returns a dictionary word_freqs

sort(word_freqs): ensures input is a non-empty dictionary, sorts by highest frequency, returns a sorted list of (word, freq) tuples.

top25_freqs(word_freqs): ensures input is a non-empty list of pairs, takes the top 25 word count pairs, formats them into a printable string.

Walkthrough of Python Code

```
#
# The main function
#
TFPassiveAggressive().bind(get_input).bind(extract_words).bind(remove_stop_words).bind(frequencies).bind(sort).bind(top25_freqs).printme()
```

- The main function chains all processing steps.
- Each function receives the output of the previous one.
- Each function in the process is called with `bind(self, func)`, allowing pipeline to stop and record the error if any step fails.
- `printme()` outputs either the final top 25 results, or the error along with the failing function.

This program processes a text file through a sequence of functions to produce the top 25 most frequent words. It uses the **passive-aggressive** programming style, where each step runs <u>only if</u> the previous ones succeeded.

Demo of Typescript Code

```typescript
#!/usr/bin/env ts-node

const fs: any = require("fs");
const path: any = require("path");


// The PassiveAggressive class for this example

class TFPassiveAggressive {
    private _e: Error | null = null;
    private _offendingFunc: Function | null = null;
    private _value: any = null;

    bind(func: Function): this {
        if (this._e === null) {
            try {
                this._value = func(this._value);
            } catch (e: any) {
                this._e = e;
                this._offendingFunc = func;
            }
        }
        return this;
    }

    printme() {
        if (this._e === null) {
            console.log(this._value);
        } else {
            console.log(`${this._e}  in  ${this._offendingFunc?.name}`);
        }
    }
}
```

Differences:

- Stronger typing with typescript

Similarities:

- Input
- Output
- Structure

Demo of Typescript Code

```typescript
// Functions translated from Python

function get_input(_: any): string {
    if (process.argv.length <= 2) {
        throw new Error("You idiot! I need an input file! I quit!");
    }
    return process.argv[2];
}

function extract_words(pathToFile: string): string[] {
    if (typeof pathToFile !== "string")
        throw new Error("I need a string! I quit!");
    if (!pathToFile)
        throw new Error("I need a non-empty string! I quit!");

    const data = fs.readFileSync(pathToFile, "utf8");
    const pattern = /[\W_]+/g;
    return data.replace(pattern, " ").toLowerCase().split(/\s+/).filter(Boolean);
}

function remove_stop_words(wordList: string[]): string[] {
    if (!Array.isArray(wordList))
        throw new Error("I need a list! I quit!");

    const stopWordsPath = path.join("./stop_words.txt");
    const stopWords: string[] = fs.readFileSync(stopWordsPath, "utf8").split(",");

    // Add single-letter words
    stopWords.push(...("abcdefghijklmnopqrstuvwxyz".split("")));

    return wordList.filter((w: string) => w && !stopWords.includes(w));
}
```

Differences:

- Stronger typing with typescript
- Typescript version uses if statements instead of assert statements
- Typescript uses `.filter()`

Similarities:

- Input
- Output
- Structure

Demo of Typescript Code

```typescript
function frequencies(wordList: string[]): Record<string, number> {
    if (!Array.isArray(wordList))
        throw new Error("I need a list! I quit!");
    if (wordList.length === 0)
        throw new Error("I need a non-empty list! I quit!");

    const wordFreqs: Record<string, number> = {};
    for (const w of wordList) {
        wordFreqs[w] = (wordFreqs[w] || 0) + 1;
    }
    return wordFreqs;
}

function sort(wordFreqs: Record<string, number>): [string, number][] {
    if (typeof wordFreqs !== "object" || Array.isArray(wordFreqs))
        throw new Error("I need a dictionary! I quit!");
    if (Object.keys(wordFreqs).length === 0)
        throw new Error("I need a non-empty dictionary! I quit!");

    // Force the tuple type so TypeScript understands entries are [string, number]
    return (Object.entries(wordFreqs) as [string, number][]).sort((a, b) => b[1] - a[1]);
}

function top25_freqs(wordFreqs: [string, number][]): string {
    if (!Array.isArray(wordFreqs))
        throw new Error("I need a list! I quit!");
    if (wordFreqs.length === 0)
        throw new Error("I need a non-empty dictionary! I quit!");

    let top25 = "";
    for (const [word, freq] of wordFreqs.slice(0, 25)) {
        top25 += `${word} - ${freq}\n`;
    }
    return top25;
}
```

Differences:

- Stronger typing with typescript
- Typescript version uses if statements instead of assert statements
- Typescript has to force the entries in wordFreqs

Similarities:

- Input
- Output
- Structure

Demo of Typescript Code

```
// Main function

new TFPassiveAggressive()
    .bind(get_input)
    .bind(extract_words)
    .bind(remove_stop_words)
    .bind(frequencies)
    .bind(sort)
    .bind(top25_freqs)
    .printme();
```

The main function is essentially identical to the Python version.

Challenges:
Minor troubleshooting with Github Copilot to fix some broken import statements at the top of the file.

Final Thoughts

Motivation

- We picked this style as it allows for clear error handling.
- By error handling at a higher level, it allows for the code to be more readable and modifiable in the lower-logic levels.
- The pinpointed error messages make it easy to trace which function the error originated from.

Problems

- Excessive validation can be overwhelming, reducing overall readability.
- Repeated input checks can feel redundant.
- Understanding the correct way to approach the monadic took time, especially when looking at how errors are handled.

Sources

Source code	https://github.com/crista/exercises-in-programming-style/tree/master
Translation to Typescript	Made with ChatGPT
Trouble shooting code	Done with Github Copilot
Typescript tuples	https://www.w3schools.com/typescript/typescript_tuples.php
Typescript .filter()	https://www.geeksforgeeks.org/typescript/typescript-array-filter-method